

CS633 Assignment 1

Group 54

Rohit Verma (220917)
Sarthak Paswan (220976)
Ved Prakash Vishwakarma (221180)
Chaitanya Vishnoi (230316)

12/04/26

1 Code Logic

The global 3D domain is partitioned into a $p_x \times p_y \times p_z$ grid of processes. Each process handles a local subdomain of size $n_x \times n_y \times n_z$, augmented with halo layers of width R , where

$$R = \frac{d-1}{6}.$$

1.1 Data Representation

The 3D data is stored in flattened 1D arrays for improved cache locality. Each process maintains:

- **data** (current state)
- **next** (updated state)

1.2 Initialization

Data is initialized using a seeded random generator. Initialization is local to each process and depends on the process rank.

1.3 Halo Exchange

Halo exchange is performed in all six directions ($\pm X, \pm Y, \pm Z$). MPI derived datatypes are used to efficiently communicate non-contiguous boundary regions. Non-blocking communication (`MPI_Isend`, `MPI_Irecv`) is used, followed by synchronization.

1.4 Stencil Computation

A d -point stencil is applied at each grid point. For $d = 7$, each point is updated using its six neighbors and itself. Computation is performed only on interior points, using halo values for boundary access.

1.5 Isovalue Counting

Instead of checking exact equality, isovalue crossings are detected by comparing neighboring values. A crossing is counted if the isovalue lies between two adjacent grid points.

1.6 Reduction

Each process computes local counts for each field. Global counts are obtained using `MPI_Reduce` at the root process.

1.7 Iteration

The process is repeated for T time steps. Double buffering is used by swapping pointers between `data` and `next`.

2 Code Optimizations

Several optimizations were applied to improve performance:

2.1 Memory Optimization

Flattened arrays improve spatial locality and cache utilization. This avoids the overhead of multi-dimensional indexing.

2.2 Communication Optimization

MPI derived datatypes (`MPI_Type_indexed`) eliminate the need for manual packing/unpacking. Non-blocking communication reduces synchronization delays.

2.3 Computation Optimization

Double buffering avoids unnecessary copying of arrays. Loop ordering ensures contiguous memory access in the innermost loop.

2.4 Reduced Overhead

Halo exchange is performed efficiently once per iteration per field. Pointer swapping minimizes memory movement.

3 Experimental Setup

The experiments were conducted using the following configurations:

3.1 Process Configurations

P	p_x	p_y	p_z
32	4	4	2
48	6	4	2
64	4	4	4
96	6	4	4

3.2 Problem Sizes

120^3 and 240^3

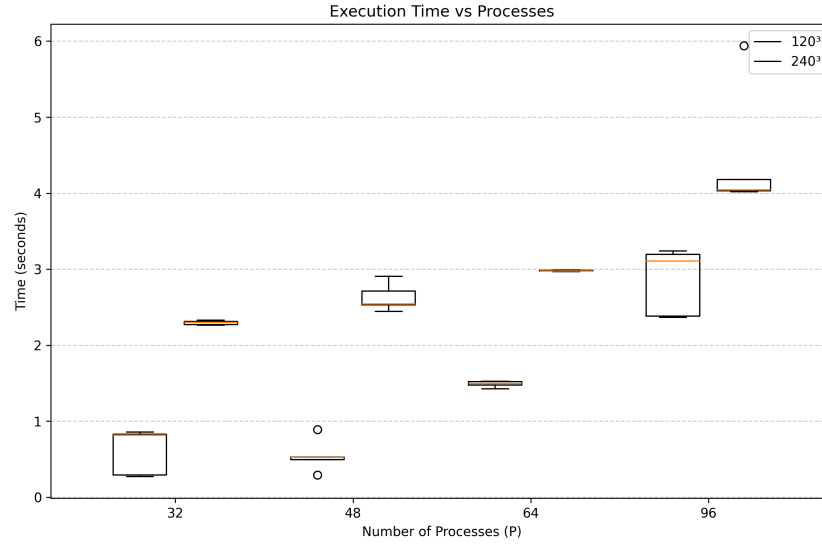
3.3 Fixed Parameters

- $d = 7$
- $T = 5$
- $F = 2$
- Seed = 1000

Each configuration was executed 5 times, and execution time was recorded.

4 Results

A boxplot was generated showing execution time versus number of processes for both problem sizes. Each box represents the distribution of execution times across 5 runs.



5 Observations

5.1 Scaling Behavior

For the 120^3 grid:

- Execution time decreases from $P = 32$ to $P = 48$
- Beyond $P = 48$, execution time increases

This indicates that the application does not scale efficiently beyond moderate process counts.

5.2 Behavior for Larger Grid (240^3)

Execution time is higher overall compared to the smaller grid. A slight improvement is observed from $P = 32$ to $P = 48$, followed by increasing runtime for larger P . The larger grid exhibits better scalability initially but still suffers at higher process counts.

5.3 Communication Overhead

As the number of processes increases:

- Subdomain size decreases
- Relative communication cost increases

Halo exchange becomes a dominant factor at higher P , limiting performance gains.

5.4 Non-Monotonic Scaling

Ideally, execution time should decrease monotonically with increasing P . However, results show:

- Improvement up to $P = 48$
- Degradation beyond that

This indicates that the strong scaling limit has been reached.

5.5 Variability Across Runs

Boxplots show low variance, indicating stable execution. A few outliers are observed, likely due to network contention and system scheduling effects.

5.6 Comparison of Problem Sizes

The 240^3 grid consistently takes more time. However, it shows relatively better scaling due to a higher computation-to-communication ratio. Despite this, communication overhead still dominates at higher process counts.

6 Conclusion

The parallel stencil implementation demonstrates correct functionality and reasonable performance scaling up to moderate process counts. However, beyond a certain number of processes, performance degrades due to increased communication overhead.

The results highlight the importance of balancing computation and communication in distributed memory systems. Optimal performance is achieved at intermediate levels of parallelism, where the computation-to-communication ratio is favorable.